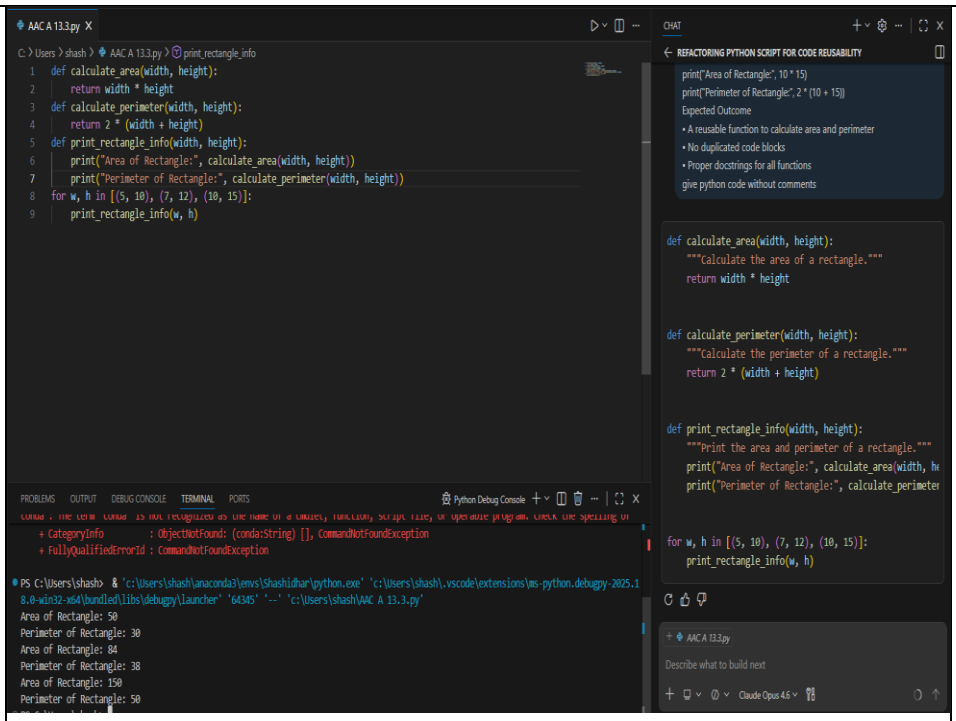
Name:B.Nithin H.No:2303A51803 Batch:26

| SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE | | DEPARTMENT OF COMPUTER SCIENCE ENGINEERING | |
|---|---|---|---|
| **Program Name:** B. Tech | | **Assignment Type: Lab** | **Academic Year:**2025-2026 |
| **Course Coordinator Name** | Dr. Rishabh Mittal | | |
| **Instructor(s) Name** | Mr. S Naresh Kumar<br>Ms. B. Swathi<br>Dr. Sasanko Shekhar Gantayat<br>Mr. Md Sallauddin<br>Dr. Mathivanan<br>Mr. Y Srikanth<br>Ms. N Shilpa<br>Dr. Rishabh Mittal (Coordinator)<br>Dr. R. Prashant Kumar<br>Mr. Ankushavali MD<br>Mr. B Viswanath<br>Ms. Sujitha Reddy<br>Ms. A. Anitha<br>Ms. M.Madhuri<br>Ms. Katherashala Swetha<br>Ms. Velpula sumalatha<br>Mr. Bingi Raju | | |
| **CourseCode** | 23CS002PC304 | **Course Title** | AI Assisted Coding |
| **Year/Sem** | III/II | **Regulation** | R23 |
| **Date and Day of Assignment** | **Week7 – Wednusday** | **Time(s)** | 23CSBTB01 To 23CSBTB52 |
| **Duration** | 2 Hours | **Applicable to Batches** | All batches |
| **Assignment Number:13.3**(Present assignment number)/**24**(Total number of assignments) | | | |

| Q.No. | Question | *Expected Time to complete* |
|---|---|---|
| 1 | **Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, and Performance**<br>**Lab Objectives**<br>The objectives of this laboratory exercise are to: | Week7 - Wednesday |

| | | |
|---|---|---|
| | • Identify common code smells and inefficiencies in legacy Python programs.<br>• Use AI-assisted coding tools to refactor code for better readability and maintainability.<br>• Apply modern Python best practices while preserving original program behavior.<br>• Analyze performance improvements achieved through refactoring. | |
| | **Learning Outcomes**<br>After completing this lab, students will be able to:<br>• Detect and eliminate code duplication in Python programs.<br>• Refactor inefficient logic using optimized data structures and constructs.<br>• Improve code quality through meaningful naming conventions and documentation.<br>• Validate refactored code using test cases and performance comparison. | |
| | **Task 1: Refactoring – Removing Code Duplication**<br>**Objective**<br>To eliminate repeated logic by extracting reusable functions.<br>**Task Description**<br>Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.<br>Starter (Legacy) Code<br># Legacy script with repeated logic<br>print("Area of Rectangle:", 5 * 10)<br>print("Perimeter of Rectangle:", 2 * (5 + 10))<br>print("Area of Rectangle:", 7 * 12)<br>print("Perimeter of Rectangle:", 2 * (7 + 12))<br>print("Area of Rectangle:", 10 * 15)<br>print("Perimeter of Rectangle:", 2 * (10 + 15))<br>**Expected Outcome**<br>• A reusable function to calculate area and perimeter<br>• No duplicated code blocks<br>• Proper docstrings for all functions | |




**Task 2: Refactoring – Optimizing Loops and Conditionals**

**Objective**

To improve performance by replacing inefficient nested loops with optimized structures.

**Task Description**

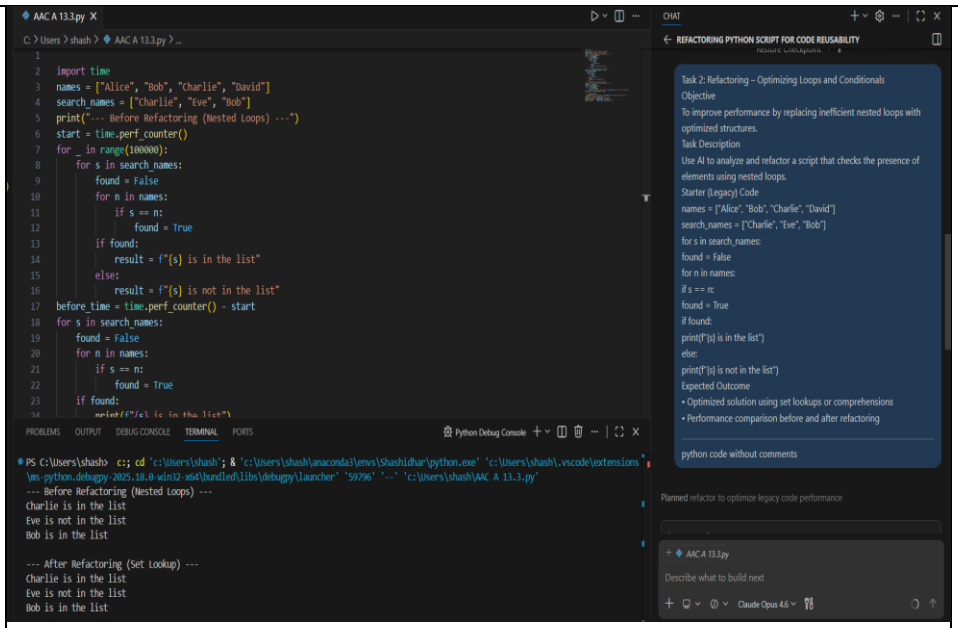
Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

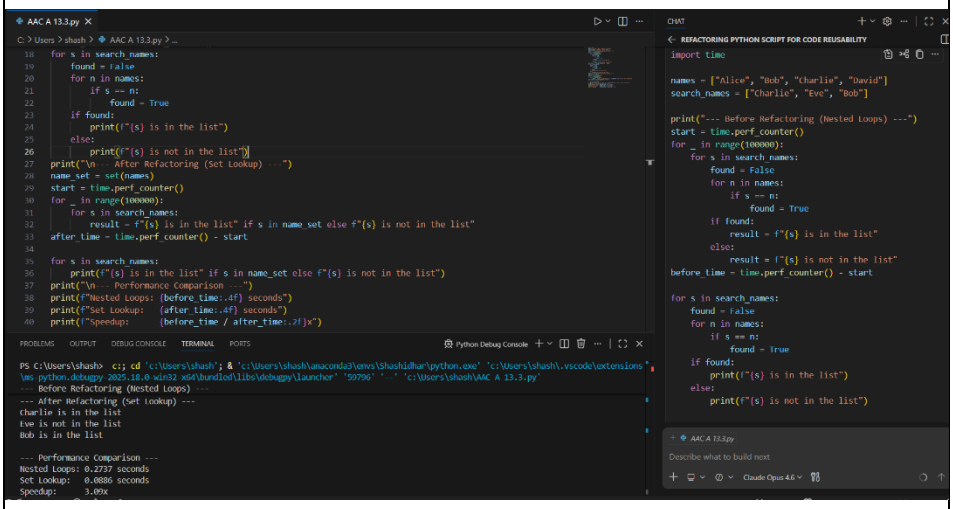
Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
```

**Expected Outcome**

- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring




**Task 3: Refactoring – Extracting Reusable Functions**

**Objective**

To modularize code by extracting calculations into reusable functions.

**Task Description**

Refactor a legacy script where price and tax calculations are written inline.

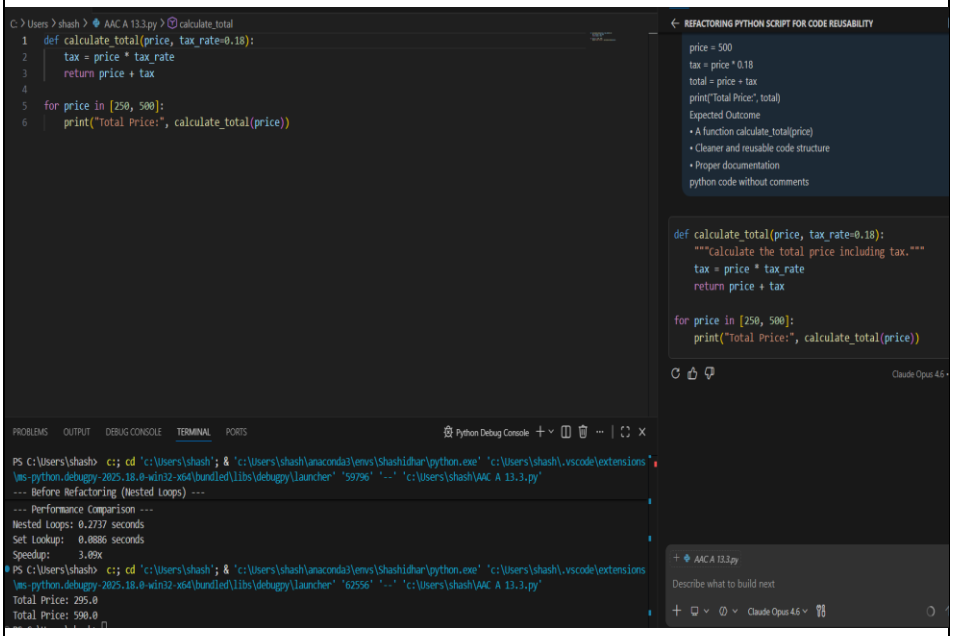
Starter (Legacy) Code

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

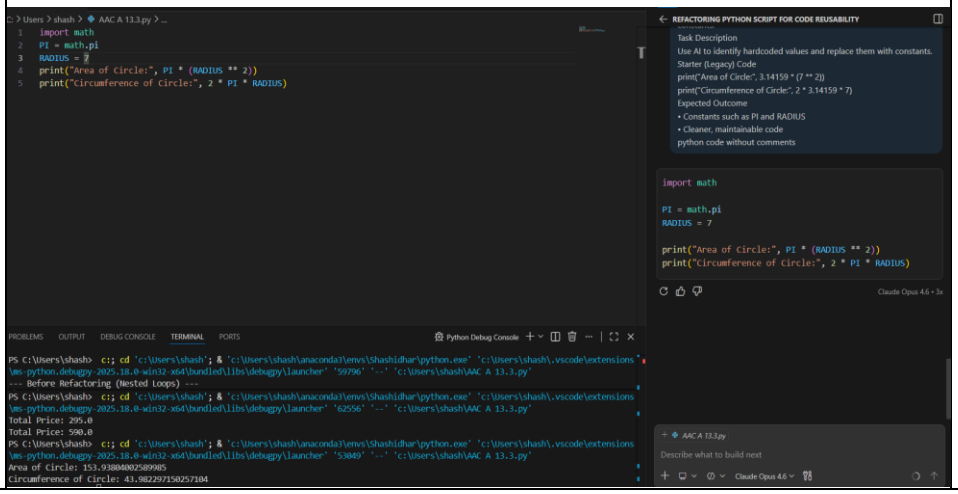
**Expected Outcome**

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation




**Task 4: Refactoring – Replacing Hardcoded Values with Constants**
**Objective**
To improve maintainability by replacing magic numbers with named constants.
**Task Description**
Use AI to identify hardcoded values and replace them with constants.
Starter (Legacy) Code
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
**Expected Outcome**

- Constants such as PI and RADIUS
- Cleaner, maintainable code



**Task 5: Refactoring – Improving Variable Naming and Readability**

**Objective**

To enhance readability using descriptive variable names and comments.

**Task Description**

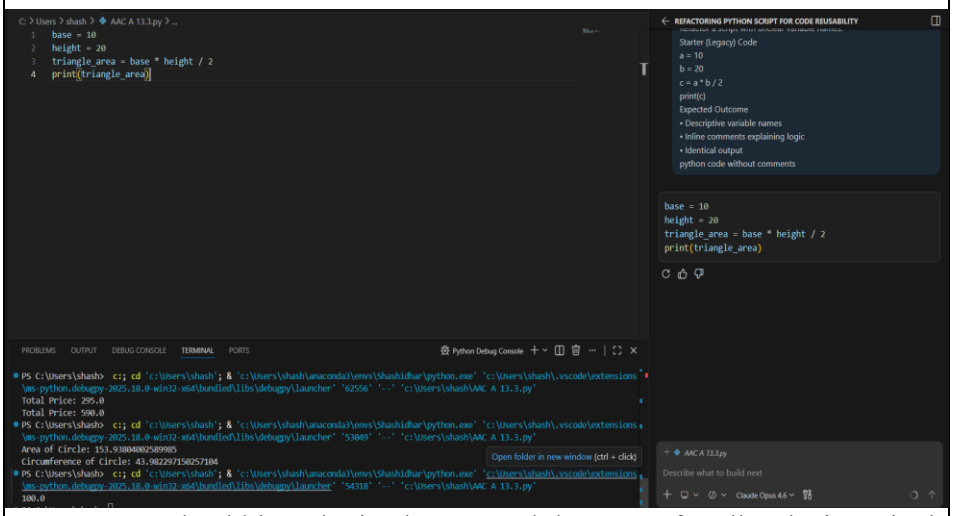
Refactor a script with unclear variable names.

Starter (Legacy) Code

```
a = 10
b = 20
c = a * b / 2
print(c)
```

**Expected Outcome**

- Descriptive variable names
- Inline comments explaining logic
- Identical output




Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.